

From Jupyter Notebooks to Workflow Graphs in Bioinformatics

Static Reconstruction of Notebook Dataflow for BioFlow-Insight

Reina
Internship report

June 2026

Abstract

Bioinformatics analyses are often described as workflows. Workflow systems such as Nextflow make these dependencies explicit and support reproducibility, reuse, and execution. In practice, however, many analyses are developed first in Jupyter notebooks.

This report presents a prototype notebook frontend for BioFlow-Insight. The prototype statically analyzes Python Jupyter notebooks, without re-executing them, and reconstructs workflow-like dependency graphs. It first builds a flat cell-level graph, then an expanded statement-level graph where each notebook cell becomes a subworkflow box and each top-level statement becomes a graph node. The implementation uses Python’s abstract syntax tree (AST) to extract definitions, uses, imports, calls, and function-body external dependencies. It handles variable redefinition through a latest-definition model and represents top-level `if` statements as conditional data producers.

1 Introduction

Bioinformatics uses computation to analyse biological data. A complete analysis is rarely one command: it is usually a chain of steps. This chain is often called a workflow. A workflow can include data cleaning, quality control, transformation, statistical modeling, visualization, and interpretation. A workflow is useful not only because it executes an analysis, but because it explains how a result was obtained and helps another person rerun, inspect, reuse, or adapt the analysis.

Workflow systems such as Nextflow were created to make these dependencies explicit and reproducible [2]. BioFlow-Insight builds on this idea by statically analyzing Nextflow workflow code and reconstructing several graph views of its structure without executing the workflow [1]. These graphs can then be read by MetroFlow, which represents workflows as metro maps: workflow steps become stations, dependencies become colored lines, and grouped regions make subworkflows visible.

Many bioinformatics analyses, however, are not initially written as formal workflows. They are often developed in Jupyter notebooks. The Jupyter Notebook format stores an ordered document made of cells, including markdown cells, code cells and saved outputs [3]. This format is useful for exploration and explanation, but the notebook interface does not make the underlying data dependencies explicit. A variable may be defined in one cell and used several cells later. A function may be declared once and called elsewhere. A result may depend on several previous cells. The visual order of the notebook is therefore not enough to understand the actual computational structure.

This internship investigated the following research question:

To what extent can static analysis recover a useful workflow-like dataflow representation from Python Jupyter notebooks, while remaining compatible with BioFlow-Insight graph outputs?

The resulting prototype follows the pipeline below:

.ipynb file → code cells → cleaned Python source → AST analysis → cell facts → dependency graphs

2 Context and Related Work

2.1 Bioinformatics workflows and BioFlow-Insight

Nextflow is a widely used workflow management system for scalable and reproducible computational workflows [2]. In a Nextflow script, workflow elements such as processes and channels are already explicit in the source code. BioFlow-Insight uses that structure as input: it statically reconstructs workflow graphs from Nextflow code and exports graph views that support reuse [1].

One target representation is the MetroFlow map. In this view, workflow steps are drawn as stations, dependencies are colored lines, and larger regions group related parts of the workflow. Figure 1 shows the style that motivated the notebook output: the goal is not only to render a graph, but to make workflow structure easier to inspect.

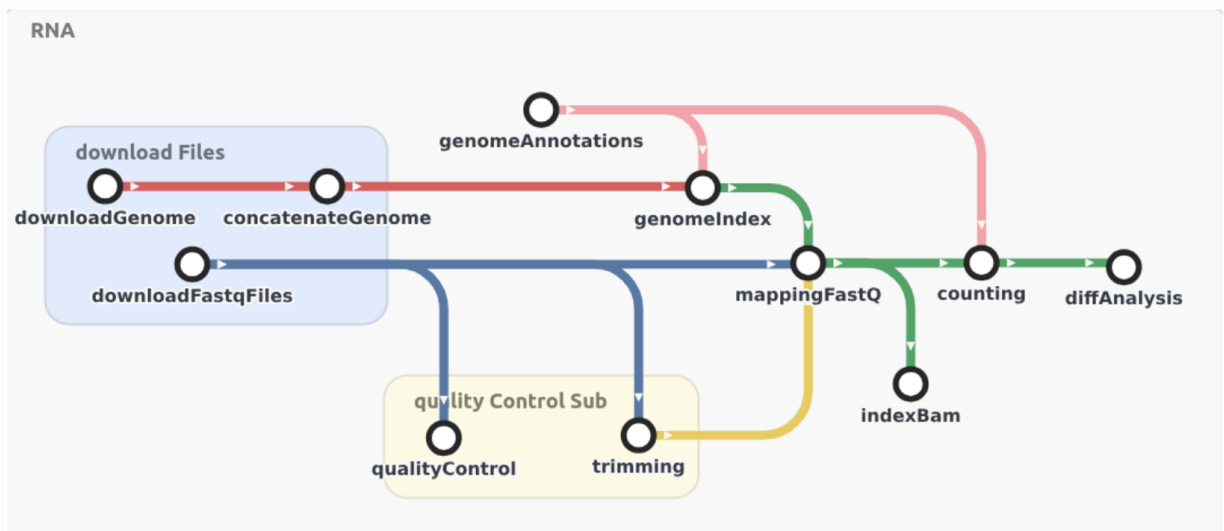


Figure 1: MetroFlow map for an RNA workflow reconstructed by BioFlow-Insight.

This project extends that idea to a less structured source format. The notebook frontend needs explicit modeling rules before it can build graphs.

2.2 Notebook dependency visualization

Notebook dependency visualization asks how to show computational structure that is hidden by the cell list. FlowBook and NBRewind use notebook dataflow mainly for reproducibility and rerun support [4, 5]. Work on notebook maintenance extracts labeled dependency graphs where code cells are nodes and edges are definition-use relations between cells; it also explicitly avoids tracking dependencies from import statements because they can obscure the graph without adding value for maintenance tasks [6]. CELLFLOW is closer to the visualization goal because it turns notebook code cells into workflow-style diagrams with cell activities, data artifacts, and data flows [9].

The specific objective here is different: the graph must remain compatible with BioFlow-Insight-style graph outputs, including JSON structures that can later be read by MetroFlow. This leads to two representations: a compact cell-level graph for process dependencies and an expanded statement-level graph where notebook cells become subworkflow groups.

3 Modeling Rules

The first step of the internship was to agree on modeling rules using small notebooks. This was necessary before analyzing larger bioinformatics notebooks, because notebooks contain many patterns that are convenient for humans but ambiguous for static analysis.

3.1 Cell selection and cleaning

The prototype focuses on Python code cells in a single notebook. It does not model dependencies across different files or across separate notebooks. The input notebook is read with `nbformat`, and each code cell is cleaned before AST parsing.

Table 1: Notebook preprocessing rules.

Case	Rule
Markdown cells	Ignored.
Empty code cells	Ignored after cleaning.
Whole-cell magics	A cell whose first non-empty line starts with <code>%%</code> is ignored.
Line magics	Lines starting with <code>%</code> are removed.
Shell commands	Lines starting with <code>!</code> are removed.
Invalid Python	If the cleaned code cannot be parsed as Python AST, the cell is stored in <code>skipped_cells</code> .
Imports	Recorded and used for filtering, but excluded from dependency edges between notebook cells.
Non-informative cells	Import-only cells, builtin-only display cells, comment-only cells, and similar cells are omitted.
Pure callable definitions	Function-only or class-only cells can be omitted as nodes, while external variables read inside function bodies are retained for later call sites.

This policy separates notebook syntax from Python syntax. For example, a line such as `%matplotlib inline` is meaningful in a notebook frontend but not valid Python for `ast.parse`. Removing it allows the ordinary Python part of the cell to be analyzed. Imports are handled separately here: they are recorded so import-only cells can be filtered and imported names can be ignored when building notebook-level data edges, following the same graph-simplification motivation used in prior notebook maintenance work [6]. The cost is that magics and shell commands are not modeled.

3.2 Cell facts

For every accepted code cell, the AST parser extracts the following facts:

Table 2: Facts stored for an accepted code cell.

Field	Meaning
<code>imports</code>	Names introduced by import statements.
<code>defines</code>	Variables, functions, classes, or unpacked names defined by the cell.
<code>callable_defines</code>	Names that can later be called, such as functions and classes.
<code>uses</code>	Names read by the cell.
<code>calls</code>	Function or method calls found in the cell.
<code>function_uses</code>	External variables read inside function bodies.

The distinction between `defines` and `uses` is the basis of the dependency graph and matches the definition-use view used in prior notebook maintenance work to derive dependencies between

code cells [6]. In `x = y + 1`, the name `x` is defined and `y` is used. In `x += y`, the name `x` is both used and redefined, because the new value depends on the old value.

4 AST-Based Analysis

Python code could be searched as plain text, but that would be fragile. The name `df`, for example, could appear in a comment, a string, an attribute, or a real variable access. The prototype therefore uses Python's `ast` module, which exposes the program as an AST, meaning an abstract syntax tree: a tree where each node represents a syntactic construct, such as an assignment, a name, a function call, a loop, a function definition, and so on. The parser walks this tree with `ast.NodeVisitor`. The official Python documentation defines AST nodes for statements, expressions, calls, names, and name contexts such as `Load`, `Store`, and `Del` [7].

For the statement:

Simple assignment used for AST analysis.

```
b = a + 1
```

a simplified AST view is:

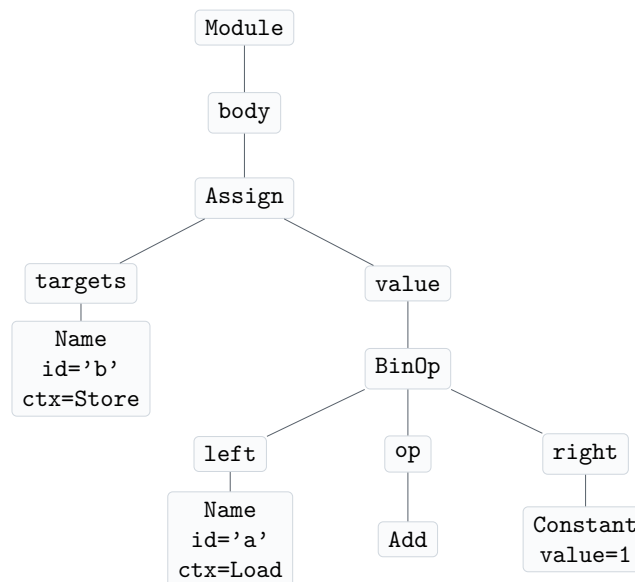


Figure 2: Simplified AST tree for `b = a + 1`.

The important information is the name context. `b` has context `Store`, so it is defined. `a` has context `Load`, so it is read.

Shortened version of the implemented AST visitor logic.

```
def visit_Assign(self, node):
    for target in node.targets:
        self.add_target_defines(target)
    self.visit(node.value)

def visit_AugAssign(self, node):
    self.add_target_uses(node.target)
    self.add_target_defines(node.target)
    self.visit(node.value)

def visit_Name(self, node):
    if isinstance(node.ctx, ast.Load):
```

```

self.add_use(node.id)

def visit_Call(self, node):
    call = self.get_call_name(node.func)
    if call is not None:
        self.add_call(call)
    self.generic_visit(node)

```

Function definitions need one additional rule, illustrated later in the controlled examples. A function name by itself does not create a data dependency edge from the definition cell to the call cell. However, if the function body reads an external variable, that external read is recorded when the function is defined and propagated to later call sites.

5 Cell-Level Dependency Graph

The first graph representation is a flat cell-level graph. Each retained code cell becomes one node. A directed edge $A \rightarrow B$ means that cell B uses data whose latest previous definition is in cell A . Before giving the formal definition, this section starts with the small notebooks that were used to decide the modeling rules.

5.1 Examples

The controlled examples were used twice. First, they were drawn by hand to decide the expected modeling behavior. Then they became real input notebooks used to check the implementation.

The first notebook contains only variable assignments and variable uses. It is the simplest case because one accepted code cell becomes one graph node, and every edge corresponds to a definition-use relation between cells.

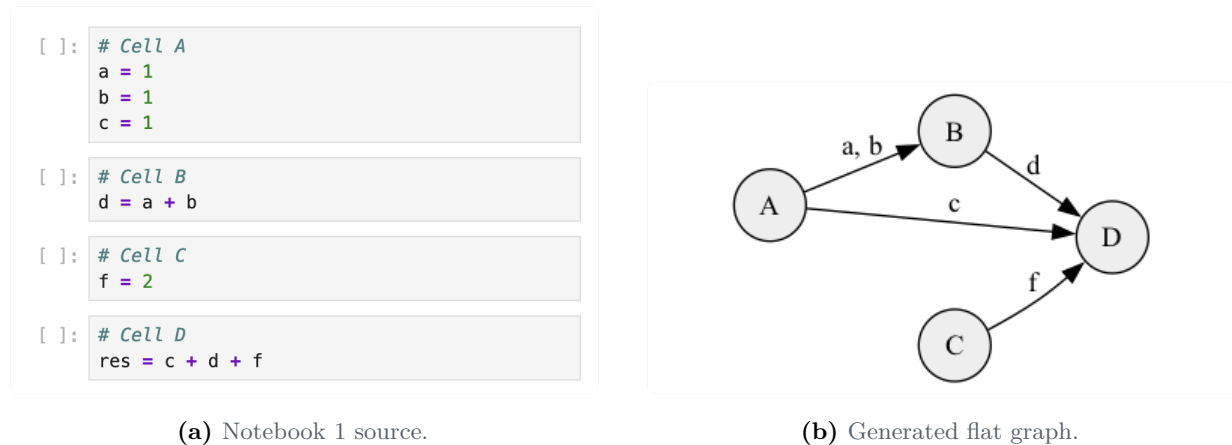


Figure 3: Notebook 1 as both design example and implemented output. The graph links the latest producers of a , b , c , d , and f to the cells that use them.

Notebook 2 introduces an import, a helper function, and a conditional call. It checks that imports and function definitions do not create dependency edges by themselves, while values passed into calls and conditions still create dependency edges.

```
[ ]: #Cell A
import random
def fun(x):
    return x+random.random()

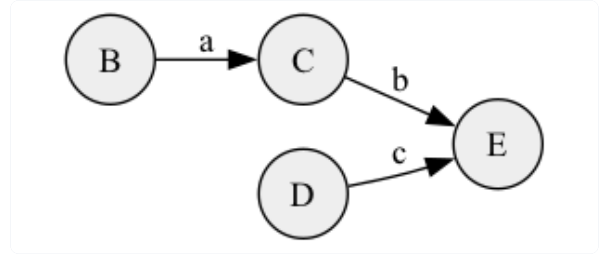
[ ]: #Cell B
a = 1

[ ]: #Cell C
b = fun(a)

[ ]: #Cell D
c = 1

[ ]: #Cell E
if(b>1.5):
    res = fun(b)
else:
    res = fun(c)
```

(a) Notebook 2 source.



(b) Generated flat graph.

Figure 4: Notebook 2 function and conditional behavior. The graph follows data arguments such as a, b, and c, but does not add a dependency edge solely from the earlier definition of fun.

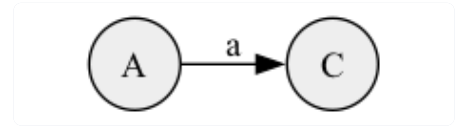
The next controlled notebook isolates external variables read inside a function body. Cell B defines `f`, whose body reads `a`; Cell C calls `f`. The generated graph contains an edge from Cell A to Cell C labeled `a`, but no edge from Cell B to Cell C solely because `f` was defined there.

```
[ ]: # Cell A
a = 10

[ ]: # Cell B
def f(x):
    return x + a

[ ]: # Cell C
b = f(2)
```

(a) Function dependency source.



(b) Generated flat graph.

Figure 5: External function-body dependency. The call cell depends on the latest producer of `a`, not on the function-definition cell by name alone.

Notebook 3 checks latest-definition behavior after redefinition.

```
[ ]: # Cell A
a = 1

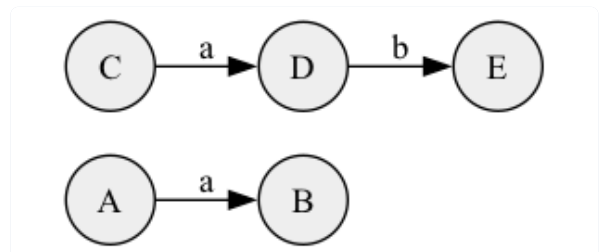
[ ]: # Cell B
b = a + 1

[ ]: # Cell C
a = 2

[ ]: # Cell D
b = a + 1

[ ]: # Cell E
c = b
```

(a) Notebook 3 source.



(b) Generated flat graph.

Figure 6: Notebook 3 latest-definition behavior. After `a` is redefined, the later assignment using `a` depends on the newer producer.

5.2 Cell-level modeling rules

From these examples, the flat cell-level graph follows the rules below:

- Markdown cells, empty cells, invalid cleaned Python cells, and non-informative display cells are not graph nodes.
- Whole-cell magics are discarded, while line magics and shell commands are removed before AST parsing.
- One accepted code cell becomes one graph node.
- Imports are recorded, but imported names are ignored when dependency edges are built.
- Pure import cells and pure callable-definition cells can be omitted as graph nodes.
- Function definitions are still analyzed before omission, so external variables read inside function bodies remain available at later call sites.
- A data edge $A \rightarrow B$ is created when cell B uses a variable whose latest previous definition is in cell A .
- Redefinition is handled with a latest-definition table: a later assignment replaces the previous producer for that name.
- A function name by itself does not create an edge from the definition cell to the call cell.
- If a called function body reads an external variable, the call cell depends on the latest producer of that external variable.
- Multiple variables transferred between the same source and target cells are merged into one edge label, such as `a, b`.

5.3 Formalization

Let $c_1, \dots, c_n$ be retained cells in notebook order. The cell-level graph is:

$$G_c = (V_c, E_c)$$

where $V_c = \{c_1, \dots, c_n\}$. For each cell c_i , the parser computes:

$$D(c_i) = \{\text{names defined in } c_i\}$$

$$U(c_i) = \{\text{names read in } c_i\}$$

$$K(c_i) = \{\text{call names found in } c_i\}$$

A forward pass over the notebook maintains a mapping L , where $L[x]$ is the most recent retained cell that defined name x . For every $u \in U(c_i)$, if u is not an ignored imported or builtin name and $L[u]$ exists, an edge $L[u] \rightarrow c_i$ labeled u is added. For every call $f \in K(c_i)$, each external variable recorded in the body of f is also looked up in L , and an edge from its latest producer to c_i is added. After all reads and calls are processed, each $d \in D(c_i)$ updates $L[d] = c_i$.

This order matters. It ensures that augmented assignments can depend on the previous value before replacing it.

Simplified cell-level edge algorithm.

```
last_def = {}
labels = {}

for cell in cells:
    for var in cell.uses:
        if var not in function_uses and var in last_def:
            add_label(labels, last_def[var], cell.id, var)

    for fun in cell.calls:
        for var in function_uses.get(fun, []):
            if var in last_def:
                add_label(labels, last_def[var], cell.id, var)

    for var in cell.defines:
        last_def[var] = cell.id
```

When several variables are transferred between the same source and target, they are merged into one edge label, such as `a, b`.

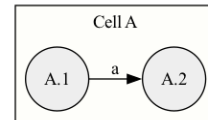
6 Statement-Level Expansion

A cell-level graph can be too coarse. In notebooks, a single cell may contain an entire analysis phase. If one graph node represents that whole cell, the user cannot see the internal dataflow between statements. For this reason, the prototype also builds an expanded statement-level graph.

For a one-cell notebook, the expanded graph contains two statement nodes inside the Cell A box, with an edge from A.1 to A.2 labeled `a`.

```
[ ]: # Cell A
a = 1
b = a
```

(a) Notebook 7 source.



(b) Generated expanded graph.

Figure 7: Expanded graph for a single cell containing two dependent statements.

This first example gives these statement-level rules:

- one accepted code cell becomes one displayed subworkflow box;
- one accepted top-level statement becomes one graph node;
- statement identifiers are stable, such as `cell_0_stmt_1`;
- statement labels are readable, such as `A.2`;
- dependencies are computed using the same latest-definition principle at statement resolution;
- top-level `if` statements create conditional producers.

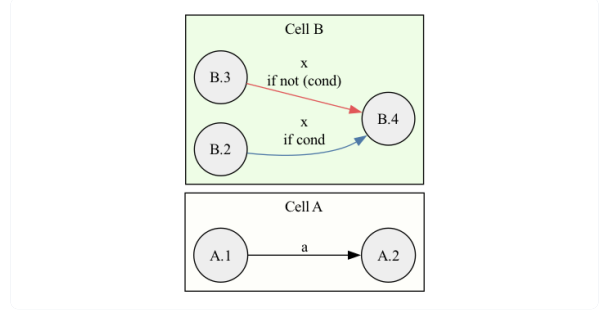
6.1 Conditional producers

Top-level `if` statements require special handling because a variable may be defined by one branch or another depending on the runtime condition. The static graph does not know which branch will execute, so it represents both possibilities.

```
[ ]: # Cell A
a = 1
cond = a > 2

[ ]: # Cell B
x = 0
if cond:
    x = 1
else:
    x = 2
y = x
```

(a) Notebook 8 source.



(b) Generated expanded graph.

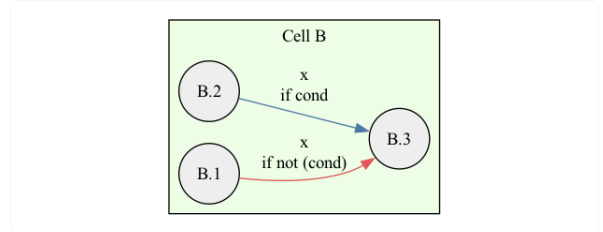
Figure 8: Conditional redefinition with an explicit `else` branch. Edge labels include both data names and branch conditions.

The statement $y = x$ may depend on the producer $x = 1$ under condition `cond`, or on $x = 2$ under condition `not (cond)`.

A missing `else` is also meaningful:

```
[ ]: # Cell A
x = 0
if cond:
    x = 1
y = x
```

(a) Notebook 9 source.



(b) Generated expanded graph.

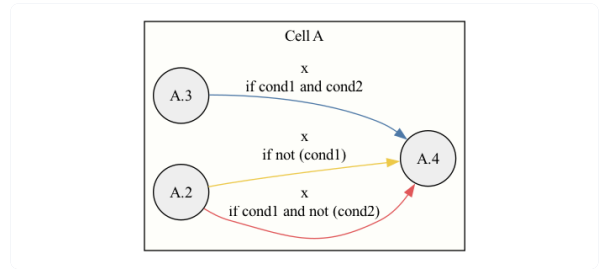
Figure 9: Conditional redefinition without an `else` branch. The false path preserves the previous producer.

Here, $y = x$ can depend on $x = 1$ when `cond` is true, but it can also depend on the older definition $x = 0$ when `cond` is false. The implementation explicitly preserves this false path.

Nested top-level `if` statements combine conditions. For example, a statement inside `if cond1` and then inside `if cond2` receives the condition `cond1 and cond2`. If a previous definition is kept because an inner condition is false, the graph can produce conditions such as `cond1 and not (cond2)`.

```
[ ]: # Cell A
cond1 = cond2 = True
x = 0
if cond1:
    if cond2:
        x = 1
y = x
```

(a) Notebook 10 source.



(b) Generated expanded graph.

Figure 10: Nested conditional example, where conditions are merged along branch paths.

6.2 Statement-level formalization

Let $s_{i,j}$ be the j -th accepted top-level statement of cell c_i . The statement graph is:

$$G_s = (V_s, E_s)$$

where V_s contains statement nodes, and cell subworkflows group the statements that came from the same original notebook cell.

For each statement s , the parser records:

$$D(s), \quad U(s), \quad K(s), \quad \kappa(s)$$

where $\kappa(s)$ is the condition under which the statement can execute. For ordinary top-level statements, $\kappa(s)$ is empty. For statements in the true branch of `if p`, $\kappa(s) = p$. For statements in the false branch, $\kappa(s) = \text{not } (p)$. Nested conditions are joined with conjunction.

The latest-definition map now stores a set of possible producers:

$$L[x] = \{(s, \kappa)\}$$

because after a branch, several statements may be possible producers for the same variable. When a later statement reads x , the graph creates one edge from each possible producer, with the merged condition attached to the edge.

Only top-level `if` structures are currently expanded as separate branch structures. Other compound statements such as `for`, `while`, `match`, `try`, and `with` are treated as one statement node at the display level, although the AST extractor still analyzes names inside them for that node.

7 Implementation

The notebook frontend is organized around small classes with separate responsibilities.

Table 3: Main implementation components.

Component	Role
<code>Notebook_File</code>	Reads the <code>.ipynb</code> file, keeps and cleans Python code cells, records skipped cells, imported names, builtins, and function-body external uses.
<code>Notebook_Cell</code>	Stores one accepted code cell and the facts extracted from it: code, output, imports, definitions, uses, calls, and callable definitions.
<code>Python_AST_Parser</code>	Walks the AST of one cell or statement and extracts definitions, uses, imports, calls, and external function-body uses.
<code>Notebook_Graph</code>	Builds the flat cell-level graph and the expanded statement-level graph. It implements latest-definition edge construction and conditional branch merging.
<code>Python_Statement_Parser</code>	Splits one accepted cell into statement nodes and branch items. It assigns stable statement identifiers and condition strings.
<code>Notebook_Statement</code>	Stores one statement node, including its code, extracted facts, and condition.

The ordinary generation pipeline is:

1. read a notebook with `Notebook_File`;
2. clean and filter Python code cells;
3. analyze each accepted cell with `Python_AST_Parser`;
4. store extracted facts in `Notebook_Cell`;
5. build dependency edges with `Notebook_Graph`;
6. export DOT, PNG, and JSON artifacts.

7.1 Output formats

The prototype emits two families of outputs. The flat cell graph is written under `output/`, and the expanded statement graph is written under `output_expanded/`. In each case, the scripts produce DOT, JSON, and PNG files.

The expanded JSON keeps the top-level shape needed for MetroFlow-style integration:

Simplified expanded JSON shape.

```
{
  "nodes": [
    {
      "id": "cell_0_stmt_0",
      "name": "A.1",
      "position": {"x": "...", "y": "..."},
      "code": "a = 1"
    }
  ],
  "edges": [
    {
      "A": "cell_0_stmt_0",
      "B": "cell_0_stmt_1",
      "color": "",
      "condition": "",
      "id": "cell_0_stmt_0 -> cell_0_stmt_1"
    }
  ],
  "subworkflows": {
    "cell_0": {
      "nodes": ["cell_0_stmt_0", "cell_0_stmt_1"],
      "label": "Cell A",
      "color": "#fefefa"
    }
  }
}
```

Graphviz is used to render DOT graphs and compute node positions [8]. The JSON output then reuses those positions, which makes the same graph structure available for visual interfaces.

7.2 MetroFlow visualization

The BioFlow-Insight MetroFlow example in Figure 1 is the visual target for the notebook frontend. For notebooks, the same visual language can be reused.

This is why the expanded notebook JSON stores `nodes`, `edges`, `position`, `color`, `condition`, and `subworkflows`. A notebook result using this visual grammar is shown later in Figure 13.

8 Integration with BioFlow-Insight

The notebook frontend was also integrated into the existing BioFlow-Insight API in a separate BioFlow-Insight repository. The goal of that integration was to make notebooks usable through the same high-level `Workflow` class as Nextflow workflows, with minimal duplication.

After integration, a notebook can be analyzed through:

Notebook use through the BioFlow-Insight API.

```
from src.workflow import Workflow
```

```
w = Workflow("../MyNotebook.ipynb")
w.initialise_with_bioflow()
w.generate_process_dependency_graph(render_graphs=False)
w.get_metro_map_json()
```

The main design choice was to reuse the existing `Workflow.graph` attribute:

Notebook graph delegation.

```
self.graph = Notebook_Graph(...)
```

This keeps notebook behavior close to existing Nextflow behavior, where `Workflow` delegates graph operations to `self.graph`. The `Workflow` class detects `.ipynb` inputs, sets `workflow_kind` to `notebook`, and branches only when notebook behavior differs from Nextflow behavior.

9 Validation

Validation was performed at two levels: controlled unit tests and generation on example notebooks.

The unit tests check the core modeling rules:

- one accepted code cell becomes one graph node;
- an edge means a later cell uses data defined earlier;
- function call names alone do not create dependency edges;
- external variables used inside called function bodies are propagated to the call cell;
- DOT, JSON, and PNG outputs are generated;
- DOT and JSON contain the same graph nodes and edges;
- expanded outputs contain statement nodes and cell subworkflows with stable identifiers.

The prototype was also run on a collection of small controlled notebooks and larger bioinformatics notebooks.

Figure 11 shows one such larger example, generated from the Day34 GWAS notebook.

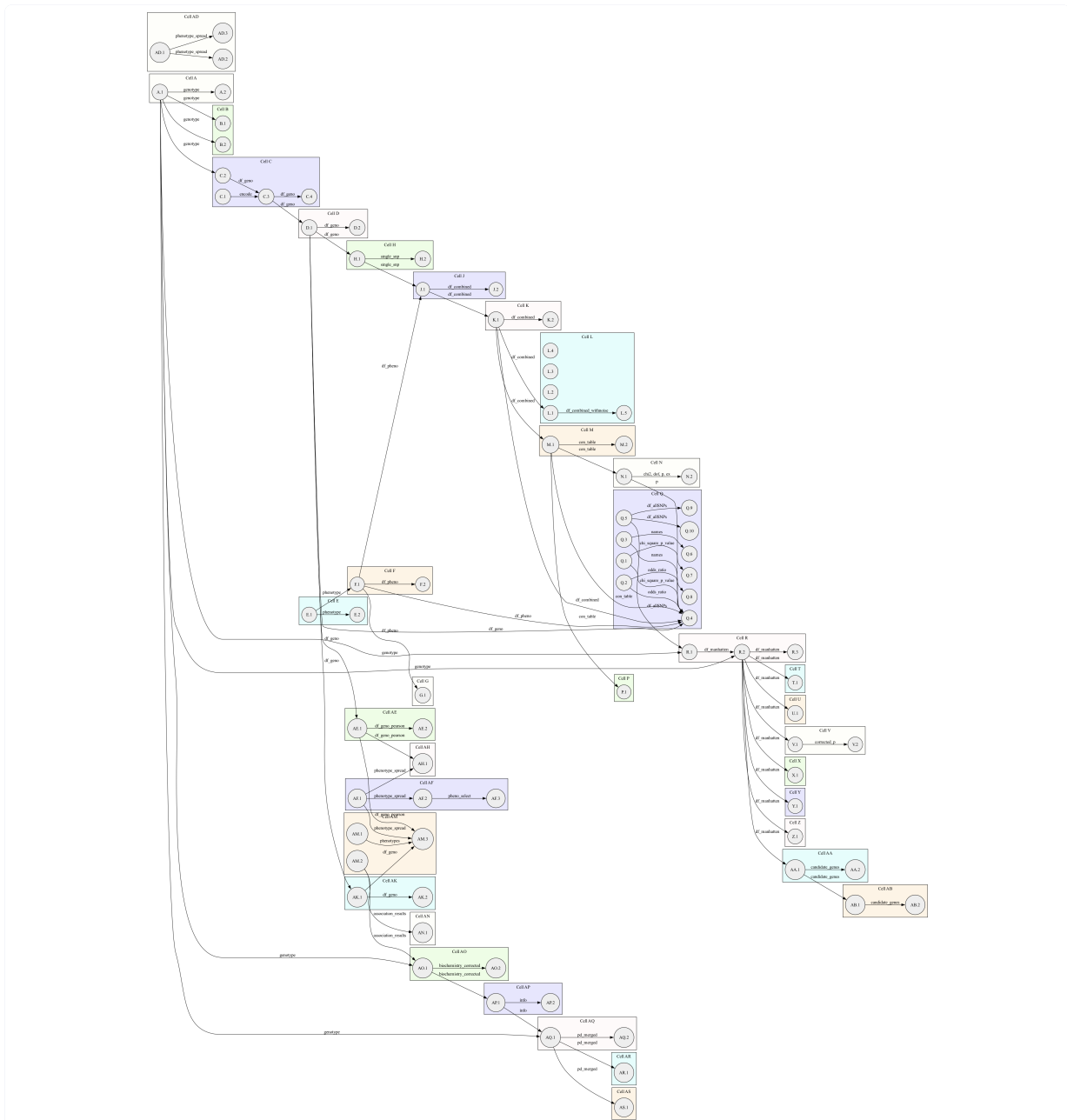


Figure 11: Expanded graph generated for the Day34 GWAS notebook, used as a larger bioinformatics example.

A second realistic example is shown in Figure 12.

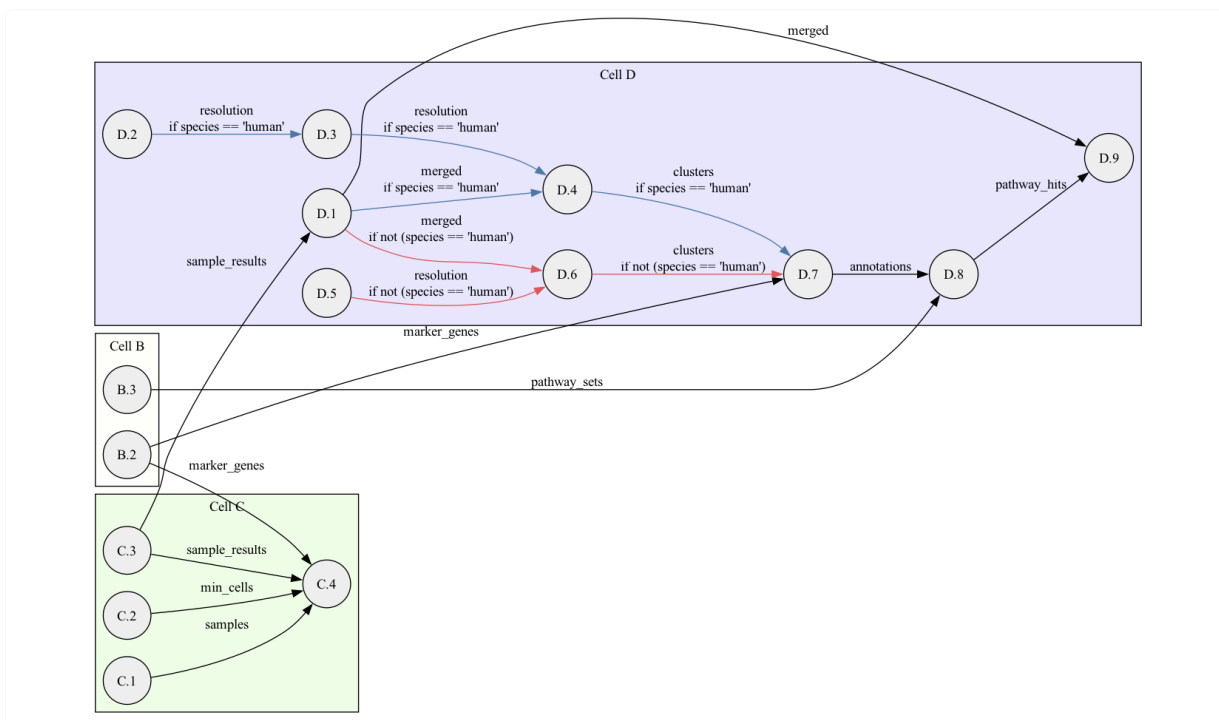


Figure 12: Expanded graph generated for the bio2 notebook, included as an additional bioinformatics example.

Finally, Figure 13 shows the same notebook graph information exported toward the MetroFlow visual style introduced earlier. The notebook cells become grouped regions, while the statement dependencies become metro lines and stations.

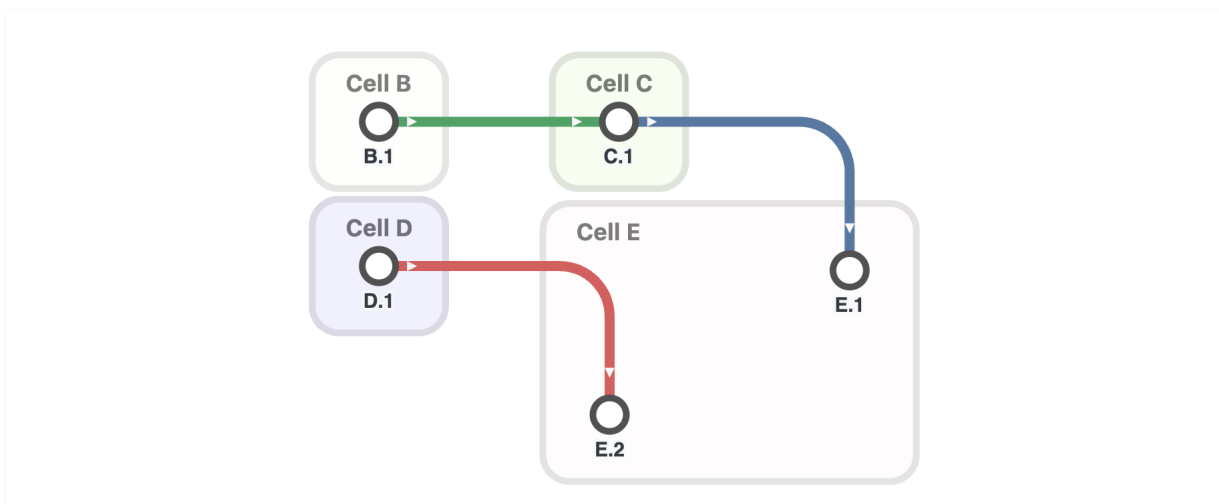


Figure 13: MetroFlow-style map produced from the notebook statement graph.

10 Limitations

The prototype is intentionally static. It does not execute notebooks and therefore cannot know runtime values, dynamic imports, values created by shell commands, or dependencies hidden behind reflection. This makes the analysis safer and faster, but also less precise than a runtime tracer.

The current scope is one Python notebook at a time. Dependencies on external files, imported modules, other notebooks, or data files are not reconstructed. Imports are recorded but deliberately

excluded from dependency edges between notebook cells, because including every imported package would make the graph harder to read without adding useful notebook-level dataflow.

The handling of conditions is limited to top-level `if` structures. Nested `if` statements are supported when they appear inside parsed branches, but loops, `try/except`, `match`, and other compound statements are not expanded into separate internal subgraphs. They are represented as statement nodes whose internal names are extracted by the AST parser.

The function model is also approximate. The prototype propagates external variables read inside function bodies to call sites, but it does not fully model side effects, mutation through aliases, object attributes, complex method calls, or data hidden in global state. Similarly, it treats notebooks as if top-to-bottom source order is the relevant order, even though Jupyter users can execute cells out of order. The resulting graph should therefore be understood as a reconstruction of the notebook as written, not a complete reconstruction of every possible interactive execution history.

11 Conclusion

This internship explored how a Python Jupyter notebook can be converted into a workflow-like graph without executing it. The prototype shows that static AST analysis can recover useful dataflow information from notebooks: variable dependencies across cells, latest definitions after reassignments, external variables used through functions, intra-cell statement dependencies, and conditional producers for top-level branches.

The main result is a notebook frontend that produces both flat and expanded graph representations while preserving compatibility with BioFlow-Insight-style graph outputs. The flat graph provides a compact overview of dependencies between cells. The expanded graph gives a more detailed view where statements are grouped inside notebook cell subworkflows and conditional edges remain visible.

The work also clarified the limits of static notebook analysis. Notebooks are flexible and sometimes messy documents, so no static model can capture every behavior without tradeoffs. Nevertheless, the prototype provides a practical basis for notebook comprehension and for future integration with workflow visualization tools. As a first research internship, the project was also an introduction to the process of turning an informal problem into modeling rules, implementation choices, validation tests, and a research question that can be discussed and extended. It strengthened my interest in continuing toward research and, in the longer term, doctoral work.

References

- [1] George Marchment, Bryan Brancotte, Marie Schmit, Frédéric Lemoine, and Sarah Cohen-Boulakia. BioFlow-Insight: facilitating reuse of Nextflow workflows with structure reconstruction and visualization. *NAR Genomics and Bioinformatics*, 6(3):lqae092, 2024. <https://doi.org/10.1093/nargab/lqae092>.
- [2] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35:316–319, 2017. <https://doi.org/10.1038/nbt.3820>.
- [3] Jupyter Development Team. The Notebook file format. https://nbformat.readthedocs.io/en/stable/format_description.html.
- [4] S. N. Freund, E. D. Berger, C. Flanagan, and E. Jun. FlowBook: Enforcing Reproducibility in Computational Notebooks. arXiv:2605.01560v1, 2026. <https://arxiv.org/abs/2605.01560>.

- [5] T. Azaz, R. Ahmad, M. S. Islam, D. Thain, and T. Malik. Efficiently Reproducing Distributed Workflows in Notebook-based Systems. arXiv:2603.26965, 2026. <https://arxiv.org/abs/2603.26965>.
- [6] Yuan Jiang, Christian Kästner, and Shurui Zhou. Elevating Jupyter Notebook Maintenance Tooling by Identifying and Extracting Notebook Structures. IEEE ICSME, 2022. <https://www.cs.cmu.edu/~ckaestne/pdf/icsme22.pdf>.
- [7] Python Software Foundation. `ast` – Abstract syntax trees. <https://docs.python.org/3/library/ast.html>.
- [8] Graphviz. Graphviz documentation. <https://graphviz.org/documentation/>.
- [9] Selin Coban, David Kierdorf, Çağatay Akpınar, and Horst Lichter. CellFlow: A Tool For Automatic Jupyter Notebook Workflow Visualization. KDA-AI 2026 preprint. https://swc.rwth-aachen.de/docs/publications/2026_KDA_AI_Coban.pdf.